

**LAPORAN EVALUASI TENGAH SEMESTER (ETS)
PROJECT-BASED LEARNING (PBL)**

Mata Kuliah: Teknologi IoT

**SISTEM AKUISISI DATA *ELECTRONIC NOSE*
(*E-NOSE*) DAN
OTA (*OVER-THE-AIR*) *FIRMWARE UPDATE*
BERBASIS ESP32-S3
DAN THINGSBOARD *CLOUD***



Dosen Pengampu:
Ahmad Radhy, S.Si., M.Si

Disusun Oleh Kelompok 7:

Faris Ahmad Holili	2042241028
Yusuf Febri Andrian	2042241043
Mohammad Eka Jauhar Ismail	2042241054
Reksa Putra Armediansyah	2042241126

**DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**

Daftar Isi

1	Pendahuluan	1
1.1	Latar Belakang	1
1.2	Tujuan	1
1.3	Ruang Lingkup	2
2	Dasar Teori	3
2.1	<i>Electronic Nose</i> dan Array Sensor Gas	3
2.2	ESP32-S3 dan <i>Firmware</i> Berbasis Rust	3
2.3	Protokol MQTT dan Platform ThingsBoard	4
2.4	Mekanisme OTA (<i>Over-The-Air</i>) <i>Firmware Update</i>	4
3	Metodologi dan Implementasi Sistem	5
3.1	Arsitektur Sistem	5
3.2	Perangkat Keras (<i>Hardware</i>)	5
3.3	Struktur Partisi <i>Flash</i>	6
3.4	Tahapan Pengembangan	6
3.4.1	Fase 1 – Inisialisasi Proyek	6
3.4.2	Fase 2 – Protokol Akuisisi 8 Sensor	6
3.4.3	Fase 3 – <i>Rewrite</i> Akuisisi Sensor Murni	7
3.4.4	Fase 4 – Penggabungan Ulang Konektivitas dan OTA	7
3.4.5	Fase 5 – <i>Debugging</i> OTA <i>End-to-End</i> di Perangkat Keras	7
3.5	Implementasi <i>Firmware</i> Akuisisi Sensor	7
3.6	Implementasi Konektivitas WiFi dan MQTT	8
3.7	Implementasi OTA <i>Firmware Update</i>	8
4	Pengujian dan Hasil	11
4.1	Verifikasi <i>Checksum</i> CRC32	12
4.2	Verifikasi Token Kredensial	13

4.3	Verifikasi Penghentian <i>Loop</i> Unduhan	13
4.4	Hasil Akhir Pengujian	13
5	Kendala dan Pembelajaran	15
5.1	Konvensi <i>Byte Order</i> yang Tidak Terdokumentasi	15
5.2	Konsistensi Kredensial pada Proses <i>Build</i>	15
5.3	Pentingnya <i>Guard</i> Perbandingan Versi	15
6	Kesimpulan dan Saran	16
6.1	Kesimpulan	16
6.2	Saran	16
	Daftar Pustaka	18

Bab 1

Pendahuluan

1.1 Latar Belakang

Kopi merupakan komoditas dengan variasi jenis dan profil aroma yang sangat beragam, sehingga identifikasi jenis/varietas kopi secara objektif menjadi topik yang relevan untuk diteliti. Salah satu pendekatan yang dapat digunakan adalah *electronic nose (E-Nose)*, yaitu sistem berbasis *array* sensor gas yang meniru cara kerja indera penciuman manusia untuk mengenali profil aroma suatu zat.

Proyek **Coffee E-Nose** ini mengembangkan sistem akuisisi data berbasis mikrokontroler ESP32-S3 yang membaca sinyal dari delapan kanal sensor gas melalui dua buah ADC eksternal ADS1115, serta sensor suhu dan kelembapan DHT22 sebagai data lingkungan pendukung. Data yang diakuisisi dikirim secara *real-time* melalui protokol MQTT ke platform IoT *cloud* **ThingsBoard**, sekaligus dicatat secara lokal melalui antarmuka serial untuk keperluan pencatatan data mentah.

Karena perangkat direncanakan digunakan pada beberapa sesi pengambilan data yang terpisah, kebutuhan untuk dapat memperbarui *firmware device* dari jarak jauh tanpa harus membongkar/mencolok ulang kabel USB menjadi penting. Oleh karena itu, sistem ini juga mengimplementasikan mekanisme **OTA (Over-The-Air) *firmware update*** yang memanfaatkan fitur bawaan ThingsBoard untuk mendistribusikan berkas *firmware* baru ke *device* secara nirkabel.

1.2 Tujuan

1. Mengimplementasikan *firmware* akuisisi data sensor gas *array* (8 kanal) dan sensor lingkungan (DHT22) pada mikrokontroler ESP32-S3 menggunakan

bahasa Rust.

2. Mengimplementasikan konektivitas WiFi dan protokol MQTT untuk mengirimkan data telemetri secara *real-time* ke ThingsBoard *Cloud*.
3. Mengimplementasikan dan menguji mekanisme OTA *firmware update end-to-end*, mulai dari proses *assign firmware* di ThingsBoard hingga *device* berhasil mengunduh, memverifikasi, dan menjalankan *firmware* baru.
4. Mendokumentasikan kendala teknis yang ditemukan selama pengujian OTA di perangkat keras (*hardware*) nyata beserta solusi yang diterapkan.

1.3 Ruang Lingkup

Laporan ini berfokus pada pengembangan sisi *firmware* (*embedded software*) dan mekanisme **OTA update**-nya. *Pipeline* pengolahan data lanjutan (ekstraksi fitur, pelatihan model klasifikasi *machine learning*) berada di luar cakupan laporan ini dan ditangani secara terpisah pada folder `04_features/` dan `05_models/` dalam struktur repositori proyek.

Bab 2

Dasar Teori

2.1 *Electronic Nose* dan *Array Sensor Gas*

Sistem *E-Nose* pada proyek ini menggunakan delapan kanal sensor gas semikonduktor *metal oxide* (MOS), yaitu MQ-3, MQ-6, MQ-7, MQ-135, TGS2600, TGS2602, TGS2611, dan TGS2620. Setiap sensor memiliki sensitivitas berbeda terhadap senyawa volatil tertentu (misalnya alkohol, gas mudah terbakar, senyawa organik volatil), sehingga kombinasi keluaran kedelapan sensor membentuk suatu *fingerprint* aroma yang dapat dibedakan antar sampel kopi.

Karena keluaran sensor berupa sinyal analog, dibutuhkan ADC (*Analog-to-Digital Converter*) beresolusi tinggi. Digunakan dua modul **ADS1115** (ADC 16-bit, antarmuka I2C, 4 kanal per modul) dengan alamat I2C berbeda (0x48 dan 0x49) sehingga total 8 kanal sensor dapat dibaca melalui satu *bus* I2C yang sama (SDA pada GPIO8, SCL pada GPIO9). Sensor DHT22 digunakan untuk mencatat suhu dan kelembapan ruangan sebagai variabel kontrol, terhubung pada GPIO10.

2.2 ESP32-S3 dan *Firmware* Berbasis Rust

ESP32-S3 dipilih sebagai mikrokontroler utama karena memiliki konektivitas Wi-Fi bawaan, memori *buffer* yang cukup untuk komunikasi MQTT, serta dukungan dua slot partisi OTA. *Firmware* ditulis dalam bahasa **Rust** menggunakan *crate* `esp-hal` (*Hardware Abstraction Layer* resmi untuk keluarga *chip* Espressif) dan *runtime* asinkron `embassy`, yang memungkinkan beberapa *task* (pembacaan sensor, komunikasi MQTT, dan lain-lain) berjalan secara konkuren tanpa RTOS (*Real-Time Operating System*) tradisional.

2.3 Protokol MQTT dan Platform ThingsBoard

MQTT (*Message Queuing Telemetry Transport*) adalah protokol *publish-subscribe* ringan yang umum digunakan pada sistem IoT. **ThingsBoard** adalah platform IoT *cloud* yang menyediakan *broker* MQTT, *dashboard* visualisasi, penyimpanan *attribute* (*client-side* dan *shared/server-side*), serta fitur manajemen OTA *firmware update* bawaan yang digunakan langsung pada proyek ini tanpa perlu membangun *server* OTA sendiri.

2.4 Mekanisme OTA (*Over-The-Air*) *Firmware Update*

Konsep OTA *update* pada ESP32 (dan *chip* Espressif pada umumnya) mengandalkan skema ***dual-partition***: terdapat dua partisi aplikasi (`ota_0` dan `ota_1`) berukuran sama, dan *device* hanya menjalankan salah satunya pada satu waktu. *Firmware* baru selalu ditulis ke partisi yang sedang tidak aktif, sehingga apabila proses unduh gagal di tengah jalan, partisi yang sedang berjalan tidak tersentuh sama sekali dan *device* tetap dapat berfungsi normal. Setelah data *firmware* baru selesai diunduh dan diverifikasi (biasanya melalui *checksum* CRC32), *bootloader* diarahkan untuk mem-*boot* dari partisi baru tersebut pada *reboot* berikutnya.

Pada proyek ini, mekanisme *low-level* penulisan ke partisi OTA ditangani oleh *crate* Rust `esp-hal-ota`, sedangkan protokol pertukaran data *firmware* antara *device* dan ThingsBoard mengikuti protokol resmi ThingsBoard berbasis topik MQTT `v1/devices/me/attributes` (untuk metadata versi) dan `v2/fw/request/{id}/c` (untuk mengunduh *firmware* per-*chunk*).

Bab 3

Metodologi dan Implementasi Sistem

3.1 Arsitektur Sistem

Secara umum, arsitektur sistem terdiri atas tiga lapisan:

1. **Lapisan akuisisi** – ESP32-S3 membaca 8 kanal sensor gas via dua ADS1115 serta satu sensor DHT22, kemudian mencetak data ke antarmuka serial dalam format CSV setiap 1 detik.
2. **Lapisan konektivitas** – ESP32-S3 terhubung ke jaringan WiFi lokal, kemudian membentuk koneksi MQTT ke `thingsboard.cloud` untuk mengirim telemetri dan menerima perintah/*attribute* (termasuk metadata OTA).
3. **Lapisan manajemen *firmware* (OTA)** – ThingsBoard menyimpan paket *firmware* pada *Packages repository*, yang dapat di-*assign* ke *device* tertentu. *Device* secara otomatis mendeteksi penetapan (*assignment*) tersebut dan mengunduhnya bila versinya berbeda dari yang sedang berjalan.

3.2 Perangkat Keras (*Hardware*)

- Mikrokontroler: ESP32-S3 (*flash* 16 MB)
- ADC eksternal: 2x ADS1115 (I2C, alamat 0x48 dan 0x49)
- Sensor gas: MQ-3, MQ-6, MQ-7, MQ-135, TGS2600, TGS2602, TGS2611, TGS2620 (total 8 kanal, terbagi merata pada 2 modul ADS1115)
- Sensor lingkungan: DHT22 (suhu & kelembapan), pada GPIO10

- *Bus* I2C: SDA = GPIO8, SCL = GPIO9

3.3 Struktur Partisi *Flash*

Agar mendukung OTA, tabel partisi *flash* dirancang dengan dua slot aplikasi seperti diperlihatkan pada Tabel 3.1.

Tabel 3.1: Tabel partisi *flash* ESP32-S3 (`partitions.csv`)

Nama	Tipe	Subtipe	Offset	Ukuran
nvs	data	nvs	0x9000	24 KB
otadata	data	ota	0xF000	8 KB
phy_init	data	phy	0x11000	4 KB
ota_0	app	ota_0	0x20000	1.5 MB
ota_1	app	ota_1	0x1A0000	1.5 MB

Partisi `ota_0` dan `ota_1` masing-masing berukuran 1,5 MB, cukup untuk menampung ukuran biner *firmware* hasil kompilasi (kurang lebih 600 KB pada proyek ini), dengan ruang cadangan untuk pengembangan fitur berikutnya.

3.4 Tahapan Pengembangan

Pengembangan *firmware* dilakukan secara bertahap dan iteratif berdasarkan riwayat *commit* pada repositori proyek. Tahapan utamanya adalah sebagai berikut.

3.4.1 Fase 1 – Inisialisasi Proyek

Struktur folder proyek, *skeleton firmware*, serta konfigurasi awal WiFi dan MQTT disiapkan. Konfigurasi kredensial (SSID, *password*, *host* ThingsBoard, dan token akses) awalnya dicoba melalui *environment variable*, kemudian dipindahkan ke berkas teks `wifi_config.txt` yang di-*gitignore* agar kredensial tidak ikut ter-*commit* ke repositori publik.

3.4.2 Fase 2 – Protokol Akuisisi 8 Sensor

Mengikuti instruksi dosen pengampu, protokol akuisisi diperbarui menjadi 8 kanal sensor gas melalui dua modul ADS1115, dengan 10 kali percobaan pengambilan

data per sesi. *Bug* kritis pada komunikasi UART yang menyebabkan *firmware hang* total juga ditemukan dan diperbaiki pada fase ini, disertai penambahan pencatatan (*logging*) lokal sebagai cadangan apabila koneksi *cloud* terputus.

3.4.3 Fase 3 – Rewrite Akuisisi Sensor Murni

Firmware ditulis ulang agar berfokus murni pada akuisisi sensor sesuai pemetaan *pin* dan format keluaran yang ditetapkan pada dokumen instruksi resmi, termasuk perbaikan *pin* DHT22 (dari asumsi awal GPIO4 menjadi GPIO10 sesuai referensi repositori acuan) dan penambahan pemindaian (*scan*) *bus* I2C saat *boot* untuk memverifikasi keberadaan perangkat pada *bus*.

3.4.4 Fase 4 – Penggabungan Ulang Konektivitas dan OTA

Setelah modul akuisisi sensor stabil, fungsi WiFi/MQTT digabungkan kembali ke dalam *firmware* yang sama, disusul dengan integrasi ulang mekanisme OTA *firmware update* menggunakan *crate esp-hal-ota* beserta tabel partisi khusus OTA.

3.4.5 Fase 5 – Debugging OTA End-to-End di Perangkat Keras

Fase ini merupakan fase pengujian paling intensif, mencakup penemuan dan perbaikan tiga kendala signifikan yang dibahas secara rinci pada Bab 4.

3.5 Implementasi *Firmware* Akuisisi Sensor

Firmware membaca kedelapan kanal sensor gas dan data DHT22 setiap 1 detik, lalu mencetak hasilnya ke antarmuka serial dalam format CSV agar mudah diproses lebih lanjut:

```
1 DATA,t_s,mq3,mq6,mq7,mq135,tgs2600,tgs2602,tgs2611,tgs2620,temp_c,
  rh_pct,dht_age_s
```

Listing 3.1: Format keluaran data akuisisi sensor

Nilai setiap kanal gas dikonversi langsung ke satuan tegangan (volt) berdasarkan LSB ADS1115 pada mode *full-scale range* $\pm 4,096$ V, sehingga keluaran lebih mudah diinterpretasikan tanpa perlu konversi manual di sisi pengolahan data.

3.6 Implementasi Konektivitas WiFi dan MQTT

Setelah koneksi WiFi berhasil dan alamat IP diperoleh melalui DHCP, *firmware* membentuk koneksi MQTT ke `thingsboard.cloud:1883` menggunakan token akses *device* sebagai kredensial (*username*). Setelah terkoneksi, *firmware*:

- (a) Men-*subscribe* topik `v1/devices/me/attributes` untuk menerima pembaruan *attribute* secara *real-time*.
- (b) Melaporkan versi *firmware* yang sedang berjalan (`current_fw_title`, `current_fw_version`) sebagai *client attribute*.
- (c) Meminta (*request*) nilai *shared attribute* terkini terkait OTA (`fw_title`, `fw_version`, `fw_size`, `fw_checksum`, `fw_checksum_algorithm`) melalui topik `v1/devices/me/attributes/request/1`.
- (d) Mempublikasikan data telemetri sensor setiap 5 detik ke topik `v1/devices/me/telemetry` disertai mekanisme *ping* berkala agar koneksi MQTT tidak terputus akibat *keep-alive timeout*.

Apabila koneksi MQTT terputus, *firmware* secara otomatis mencoba menghubungkan kembali (*reconnect*) setelah jeda 3 detik, sementara *task* akuisisi sensor dan pencatatan lokal tetap berjalan tanpa terganggu.

3.7 Implementasi OTA *Firmware Update*

Alur lengkap mekanisme OTA yang diimplementasikan adalah sebagai berikut:

1. *Device* melaporkan versi *firmware* yang sedang berjalan sebagai *client attribute* (`current_fw_title` / `current_fw_version`).
2. *Device* meminta nilai *shared attribute* terkait *firmware* yang ditetapkan (*assigned*) untuknya di ThingsBoard.
3. *Device* membandingkan versi yang ditetapkan dengan versi yang sedang berjalan. Apabila sama, proses dihentikan (*skip*); apabila berbeda, proses unduh dimulai.

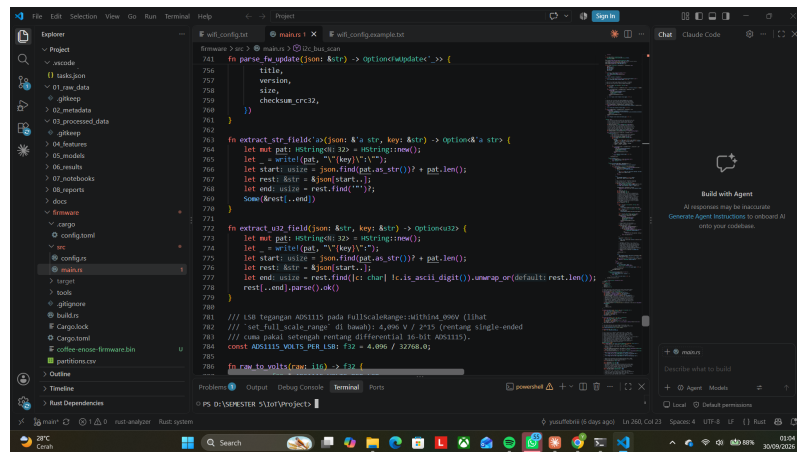
4. *Firmware* diunduh secara bertahap per-*chunk* berukuran 4096 *byte* melalui topik `v2/fw/request/{id}/chunk/{n}`, dan setiap *chunk* yang diterima langsung ditulis ke partisi OTA yang tidak aktif menggunakan *crate* `esp-hal-ota`.
5. Nilai CRC32 dihitung secara independen atas seluruh data yang diunduh, lalu dibandingkan dengan nilai *checksum* yang dikirim ThingsBoard.
6. Apabila *checksum* cocok, data *firmware* di-*flush* secara final ke *flash* dan *device* melakukan *restart* untuk berpindah menjalankan *firmware* baru dari partisi yang baru saja ditulis.

Potongan kode berikut menunjukkan bagian pengecekan versi yang ditambahkan untuk mencegah pengunduhan ulang *firmware* yang sama (dibahas lebih lanjut pada Bab 4):

```
1 if let Some(update) = parse_fw_update(text) {  
2     if update.title == CURRENT_FW_TITLE  
3         && update.version == CURRENT_FW_VERSION  
4     {  
5         info!(  
6             "OTA: {} v{} sudah versi yang jalan sekarang, skip  
7                 download",  
8             update.title, update.version  
9         );  
10        continue;  
11    }  
12    info!(  
13        "OTA package terdeteksi: {} v{} ({} bytes)",  
14        update.title, update.version, update.size  
15    );  
16    // ... proses unduh chunk demi chunk ...  
17 }
```

Listing 3.2: Pengecekan versi sebelum memulai unduhan OTA (`main.rs`)

Implementasi lengkap struktur program *firmware* pada berkas `main.rs` diperlihatkan pada Gambar 3.1.



Gambar 3.1: Tangkapan layar kode *firmware* utama (`main.rs`) berbasis Rust dan Embassy.

Bab 4

Pengujian dan Hasil

Pengujian OTA dilakukan langsung pada perangkat keras (*hardware*, bukan simulasi), dengan cara meng-*assign* paket *firmware* baru ke *device* melalui *dashboard* ThingsBoard dan mengamati proses unduhan melalui *serial monitor* (`espflash monitor`). Pada iterasi pengujian pertama, ditemukan tiga kendala signifikan yang harus diselesaikan sebelum OTA dapat berfungsi secara andal. Ringkasannya diperlihatkan pada Tabel 4.1.

Tabel 4.1: Ringkasan kendala OTA yang ditemukan dan solusinya

No	Gejala	Penyebab	Solusi
1	Verifikasi gagal / <i>checksum CRC32</i> tidak cocok	ThingsBoard mengirim nilai <i>checksum fw_checksum</i> dengan urutan <i>byte (endianness)</i> terbalik dibanding hasil perhitungan CRC32 di <i>firmware</i>	Membalik urutan <i>byte</i> nilai <i>checksum</i> menggunakan <code>.swap_bytes()</code> sebelum dibandingkan
2	<i>Device offline</i> (BadUserNameOrPassword) setelah OTA berhasil dan <i>reboot</i>	Paket <i>firmware</i> OTA yang di-build menyertakan token akses MQTT yang berbeda (kedaluwarsa) dari token yang sedang aktif di ThingsBoard, karena <i>wifi_config.txt</i> sudah berubah sejak <i>firmware</i> tersebut dikompilasi	Membangun ulang (<i>re-build</i>) paket <i>firmware</i> OTA menggunakan <i>wifi_config.txt</i> yang terbaru sebelum diunggah sebagai paket OTA baru
3	<i>Firmware</i> yang sama diunduh ulang terus-menerus setiap <i>device</i> melakukan <i>reconnect</i> MQTT (<i>infinite loop</i>)	<i>Firmware</i> tidak pernah membandingkan versi <i>firmware</i> yang ditetapkan (<i>assigned</i>) dengan versi yang sedang berjalan – setiap menerima <i>attribute fw_title/fw_version</i> , unduhan langsung dimulai tanpa syarat	Menambahkan pengecekan kesamaan judul dan versi <i>firmware</i> sebelum memulai proses unduh; unduhan di- <i>skip</i> apabila versi sudah sama

4.1 Verifikasi *Checksum CRC32*

Nilai CRC32 dihitung secara manual dan independen di sisi *firmware* atas seluruh data yang diunduh, kemudian dibandingkan dengan nilai target dari ThingsBoard. Contoh luaran *log* yang menunjukkan proses ini berhasil:

```

1 INFO - OTA: chunk 151 diterima (1024 bytes, total 619520/619520
   bytes)
2 INFO - OTA: download selesai. CRC32 kita: 0x571B2CAE,
3     target dari ThingsBoard: 0x571B2CAE (COCOK)
4 INFO - OTA sukses, reboot...

```

Listing 4.1: Cuplikan log verifikasi CRC32 yang berhasil

4.2 Verifikasi Token Kredensial

Untuk memastikan paket *firmware* yang diunggah ke ThingsBoard benar-benar menggunakan kredensial (token akses MQTT) yang aktif, dilakukan pemeriksaan *string* biner yang tertanam pada berkas `.bin` hasil kompilasi sebelum diunggah, serta verifikasi kesamaan *byte-per-byte* antara berkas lokal dan berkas yang tersimpan di ThingsBoard (dengan mengunduh kembali paket yang telah diunggah dan membandingkannya). Langkah ini terbukti efektif mendeteksi kasus ketidaksesuaian token pada iterasi pengujian sebelumnya.

4.3 Verifikasi Penghentian *Loop* Unduhan

Setelah perbaikan pada Tabel 4.1 poin 3 diterapkan, pengujian ulang menunjukkan bahwa *device* berhenti mengunduh ulang *firmware* yang versinya sama:

```

1 INFO - MQTT connected ke ThingsBoard
2 INFO - OTA: coffee-nose v2.1.2 sudah versi yang jalan sekarang,
3     skip download

```

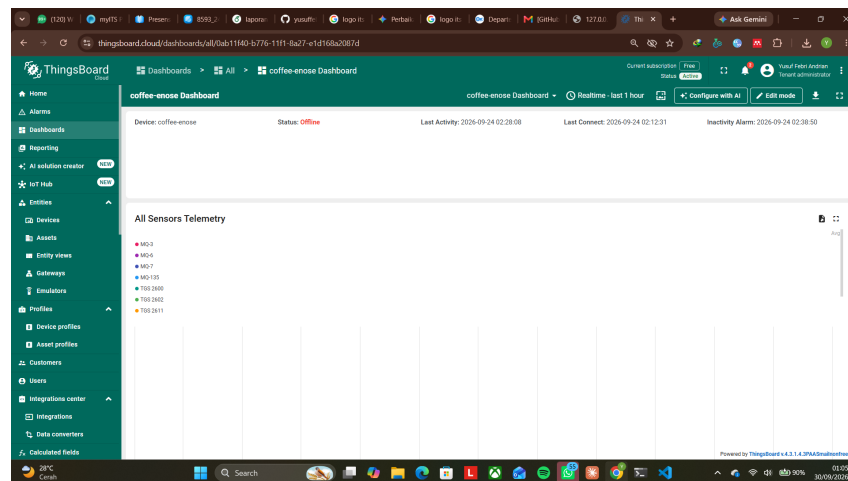
Listing 4.2: Log setelah perbaikan – unduhan otomatis di-skip

4.4 Hasil Akhir Pengujian

Setelah ketiga perbaikan pada Tabel 4.1 diterapkan, mekanisme OTA terverifikasi berjalan secara utuh (*end-to-end*) pada perangkat keras (*hardware*) sebagai berikut: penetapan (*assignment*) *firmware* baru di ThingsBoard → *device* mendeteksi perbedaan versi → unduhan berjalan per-*chunk* (151 *chunk*, total 619.520 *byte*) → verifikasi CRC32 cocok → *device* melakukan *restart* → *device* kembali terhubung

ke MQTT dengan kredensial yang tetap valid → pada percobaan koneksi berikutnya, unduhan tidak terulang karena versi telah sesuai.

Tampilan antarmuka *dashboard* ThingsBoard Cloud saat pemantauan telemetry dan status OTA *device* dapat dilihat pada Gambar 4.1.



Gambar 4.1: Tampilan *Dashboard* ThingsBoard Cloud untuk pemantauan data telemetry dan status OTA *firmware update*.

Bab 5

Kendala dan Pembelajaran

5.1 Konvensi *Byte Order* yang Tidak Terdokumentasi

Salah satu pembelajaran penting dari proyek ini adalah pentingnya melakukan verifikasi empiris terhadap asumsi format data dari pihak ketiga (dalam hal ini, representasi *byte* dari nilai *checksum* yang dikirim ThingsBoard), karena perbedaan konvensi semacam ini tidak selalu terdokumentasikan secara eksplisit pada dokumentasi resmi.

5.2 Konsistensi Kredensial pada Proses *Build*

Karena kredensial WiFi/MQTT ditanamkan langsung ke dalam biner *firmware* saat proses kompilasi (*compile-time embedding*), setiap kali paket *firmware* baru akan diunggah sebagai paket OTA, perlu dipastikan bahwa proses *build* dilakukan menggunakan berkas konfigurasi kredensial (`wifi_config.txt`) yang paling mutakhir, untuk menghindari *device* kehilangan konektivitas setelah pembaruan.

5.3 Pentingnya *Guard* Perbandingan Versi

Tanpa mekanisme pembandingan versi yang eksplisit, sistem OTA rentan terhadap kondisi *infinite loop* yang tidak perlu, membebani *bandwidth* dan siklus tulis *flash* memori *device* secara berulang. Penambahan pengecekan versi sederhana terbukti cukup untuk mencegah kondisi ini tanpa mengorbankan fungsi utama OTA.

Bab 6

Kesimpulan dan Saran

6.1 Kesimpulan

1. *Firmware* akuisisi data sensor gas *array* (8 kanal via 2x ADS1115) dan sensor lingkungan DHT22 pada ESP32-S3 berhasil diimplementasikan menggunakan Rust dengan *framework* *esp-hal* dan *embassy*.
2. Konektivitas WiFi dan MQTT ke ThingsBoard *Cloud* berhasil diimplementasikan dengan mekanisme *reconnect* otomatis dan pengiriman telemetri berkala.
3. Mekanisme OTA *firmware update* berhasil diimplementasikan dan diverifikasi berfungsi secara *end-to-end* pada perangkat keras (*hardware*) nyata, setelah melalui proses *debugging* tiga kendala teknis signifikan: perbedaan urutan *byte checksum* CRC32, inkonsistensi token kredensial pada paket OTA, dan ketiadaan pengecekan versi yang menyebabkan *infinite loop* unduhan.
4. Sistem OTA yang dihasilkan bersifat aman terhadap kegagalan (*fail-safe*): apabila proses unduh gagal di tengah jalan, partisi *firmware* yang sedang aktif tidak tersentuh, sehingga *device* tetap dapat beroperasi dengan *firmware* sebelumnya.

6.2 Saran

1. Melakukan pemasangan fisik seluruh sensor (2x ADS1115 dan DHT22) ke perangkat keras dan memvalidasi ulang seluruh jalur akuisisi data sebelum pengambilan data sampel kopi yang sesungguhnya dimulai.

2. Melakukan kalibrasi masing-masing kanal sensor gas terhadap sampel referensi yang diketahui sebelum data digunakan untuk pelatihan model klasifikasi.
3. Menambahkan mekanisme *rollback* otomatis pada *firmware* apabila *firmware* hasil OTA gagal terhubung ke jaringan setelah sejumlah percobaan tertentu, sebagai lapisan keamanan tambahan di luar verifikasi *checksum* CRC32.
4. Mendokumentasikan prosedur pembaruan OTA (termasuk langkah verifikasi *checksum* dan token) sebagai panduan baku bagi anggota tim lain yang akan melakukan pembaruan *firmware* di kemudian hari.

Daftar Pustaka

- [1] Espressif Systems, *esp-hal: Hardware Abstraction Layer for Espressif Chips*, <https://github.com/esp-rs/esp-hal>.
- [2] Embassy Project, *embassy: Async Embedded Framework for Rust*, <https://embassy.dev>.
- [3] ThingsBoard, *ThingsBoard OTA Updates Documentation*, <https://thingsboard.io/docs/user-guide/ota-updates/>.
- [4] esp-hal-ota crate documentation, <https://crates.io/crates/esp-hal-ota>.
- [5] OASIS, *MQTT Version 3.1.1 Protocol Specification*.
- [6] Texas Instruments, *ADS1115 16-Bit ADC Datasheet*.
- [7] Aosong Electronics, *DHT22 (AM2302) Digital Temperature and Humidity Sensor Datasheet*.
- [8] Repositori GitHub Proyek Coffee E-Nose IoT, <https://github.com/yusuffebriii/Coffee-Enose-IoT-Project>.